

Relatório – Trabalho 2: Telemetria em Switches P4

Disciplina: Redes de Computadores Avançadas

Universidade: UFSM – Departamento de Computação Aplicada

1. Introdução

Este trabalho apresenta a implementação de um mecanismo de telemetria em switches programáveis utilizando a linguagem P4 e o modelo BMv2 (`simple_switch`). O objetivo principal foi coletar métricas diretamente no plano de dados, exportando informações de tráfego periodicamente para um host coletor sem interferir no encaminhamento normal dos pacotes.

A solução utiliza o mecanismo de clonagem de pacotes do `v1model`, permitindo que amostras contendo informações de telemetria sejam enviadas a um coletor dedicado enquanto o pacote original continua normalmente até seu destino.

2. Abordagem Escolhida

A solução implementa a técnica de **Clone-to-Collector**, baseada em clonagem de pacotes para um controlador/coletor de telemetria.

A cada janela de **N = 10 pacotes IPv4** processados no ingress do switch, o plano de dados realiza as seguintes etapas:

1. Coleta métricas utilizando registradores P4;
2. Armazena os valores em campos de metadata anotados com `@field_list`;
3. Executa a função:

```
clone_preserving_field_list(CloneType.I2E, 99, 1);
```

4. Gera uma cópia do pacote direcionada ao host coletor `h3`;
5. No egress, adiciona um cabeçalho customizado de telemetria com EtherType `0x9999`;
6. Encaminha o pacote clonado ao coletor;

7. Permite que o pacote original continue normalmente para o destino final.

Essa abordagem foi escolhida pelas seguintes vantagens:

- Compatibilidade nativa com o modelo `v1model`;
- Simplicidade de implementação no BMv2;
- Não exige alterações nos hosts finais;
- Toda a telemetria é produzida diretamente no plano de dados P4.

3. Topologia da Rede

A topologia utilizada contém dois hosts de tráfego (`h1` e `h2`), um switch P4 (`s1`) e um host coletor (`h3`).

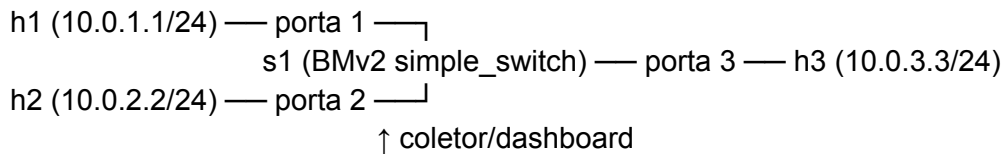


Tabela de nós

Nó	Endereço IP	MAC	Função
h1	10.0.1.1/24	00:00:00:00:01:01	Gerador de tráfego
h2	10.0.2.2/24	00:00:00:00:02:02	Receptor de tráfego
s1	—	—	Switch P4 (BMv2)
h3	10.0.3.3/24	00:00:00:00:03:03	Coletor de telemetria

A topologia é criada pelo script `topo_trabalho2.py`, responsável por:

- Instanciar os nós no Mininet;
- Inicializar o `simple_switch`;
- Carregar o JSON compilado do programa P4;
- Instalar automaticamente as regras de encaminhamento via `simple_switch_CLI`.

4. Modificações no Programa P4

4.1 Cabeçalho de Telemetria (`telemetry_t`)

Foi criado um cabeçalho customizado inserido nos pacotes clonados:

Campo	Bits	Descrição
<code>pkt_count</code>	32	Número de pacotes na janela
<code>byte_count</code>	32	Quantidade de bytes observados
<code>min_ttl</code>	8	Menor TTL observado
<code>dominant_protocolo</code>	8	Protocolo dominante
<code>reserved</code>	16	Campo reservado/alinhamento

Esse cabeçalho é encapsulado utilizando EtherType `0x9999`.

4.2 Metadata com `@field_list`

Os valores exportados ao clone são armazenados em metadata preservado pela clonagem:

```
struct metadata_t {  
    @field_list(FL_TELEM) bit<32> telem_pkt_count;  
    @field_list(FL_TELEM) bit<32> telem_byte_count;  
    @field_list(FL_TELEM) bit<8> telem_min_ttl;  
    @field_list(FL_TELEM) bit<8> telem_dominant_proto;  
}
```

A anotação `@field_list` garante que os dados sejam copiados corretamente para o pacote clonado no mecanismo `clone_preserving_field_list`.

4.3 Registradores Utilizados

Foram utilizados registradores no ingress para manter as estatísticas da janela atual.

Registrador	Tamanho	Função
-------------	---------	--------

<code>reg_pkt_count</code>	1 × 32 bits	Contador de pacotes
<code>reg_byte_count</code>	1 × 32 bits	Contador de bytes
<code>reg_min_ttl</code>	1 × 8 bits	Menor TTL observado
<code>reg_proto_count</code>	256 × 32 bits	Contador por protocolo IP

4.4 Lógica da Janela

A lógica de telemetria funciona da seguinte forma:

1. Cada pacote IPv4 recebido atualiza os registradores;
 2. O switch contabiliza:
 - quantidade de pacotes;
 - volume de bytes;
 - menor TTL;
 - quantidade de pacotes por protocolo;
 3. Quando `pkt_count == WINDOW_SIZE (10)`:
 - os contadores dos protocolos ICMP, TCP e UDP são analisados;
 - o protocolo dominante é identificado;
 - os valores são copiados para o metadata;
 - é criado um clone para o coletor;
 - os registradores são reinicializados.
-

4.5 Construção do Pacote de Telemetria no Egress

No pipeline de egress, o switch identifica os clones de ingress e adiciona o cabeçalho de telemetria:

```
if (std_meta.instance_type == 1) {
    hdr.telemetry.setValid();

    hdr.telemetry.pkt_count    = meta.telem_pkt_count;
    hdr.telemetry.byte_count   = meta.telem_byte_count;
    hdr.telemetry.min_ttl      = meta.telem_min_ttl;
    hdr.telemetry.dominant_proto = meta.telem_dominant_proto;

    hdr.ethernet.etherType = 0x9999;
}
```

5. Métricas Exportadas

5.1 Métricas Obrigatórias

Métrica	Campo P4	Descrição
Pacotes por janela	<code>pkt_count</code>	Quantidade de pacotes
Bytes por janela	<code>byte_count</code>	Volume total de dados

5.2 Métricas Adicionais

TTL mínimo (`min_ttl`)

O campo TTL diminui a cada salto de roteamento. Valores reduzidos podem indicar:

- rotas longas;
- loops de roteamento;
- ataques de TTL-expiry.

O switch registra o menor TTL observado durante toda a janela, permitindo identificar situações críticas.

Protocolo dominante (`dominant_proto`)

Identifica o protocolo de camada 4 predominante na janela observada:

Valor	Protocolo
1	ICMP
6	TCP
17	UDP

Essa métrica é útil para:

- caracterização de tráfego;
 - monitoramento de QoS;
 - identificação de anomalias;
 - detecção de floods ICMP/UDP;
 - diferenciação entre tráfego de ping e iperf.
-

6. Janela de Observação

A solução utiliza uma janela baseada em número de pacotes:

- **N = 10 pacotes IPv4**

A cada 10 pacotes observados no ingress, um relatório de telemetria é exportado.

As principais justificativas para essa escolha são:

- independência de temporizadores;
- simplicidade de implementação;
- comportamento determinístico;
- rápida visualização das mudanças de tráfego.

O tamanho da janela pode ser alterado modificando a constante:

WINDOW_SIZE

no programa P4.

7. Compilação e Execução

7.1 Compilação do Programa P4

```
mkdir -p build
```

```
p4c --target bmv2 --arch v1model \  
--p4runtime-files build/telemetria_trabalho2.p4.p4info.txt \  
-o build/ telemetria_trabalho2.p4
```

7.2 Inicialização da Topologia

```
sudo python3 topo_trabalho2.py
```

7.3 Inicialização do Dashboard

Na CLI do Mininet:

```
mininet> xterm h3
```

No terminal do host **h3**:

```
sudo python3 dashboard.py --iface h3-eth0
```

Alternativamente, pode-se utilizar o coletor em modo texto:

```
mininet> h3 sudo python3 collector.py --iface h3-eth0 &
```

7.4 Geração de Tráfego

Teste ICMP

```
mininet> h1 ping -c 50 10.0.2.2
```

Teste UDP

```
mininet> h2 iperf -u -s &  
mininet> h1 iperf -u -c 10.0.2.2 -t 30 -b 1M
```

Teste TCP

```
mininet> h2 iperf -s &  
mininet> h1 iperf -c 10.0.2.2 -t 30
```

8. Logs e Resultados

8.1 Saída do Coletor

Janela #0001 [2026-05-04 14:32:01]

Pacotes na janela : 10
Bytes na janela : 980
Tamanho medio (bytes) : 98.0
TTL minimo observado : 63
Protocolo dominante : ICMP (1)

Janela #0002 [2026-05-04 14:32:05]

Pacotes na janela : 10
Bytes na janela : 14820
Tamanho medio (bytes) : 1482.0
TTL minimo observado : 63
Protocolo dominante : UDP (17)

8.2 Verificação dos Clones

A captura dos pacotes clonados pode ser validada utilizando:

```
h3 tcpdump -i h3-eth0 -n -e ether proto 0x9999
```

O resultado esperado é a recepção de um pacote de telemetria a cada 10 pacotes trafegados entre **h1** e **h2**.

9. Análise dos Resultados

Os resultados obtidos demonstram o funcionamento correto da solução.

Cenário ICMP (ping)

- Pacotes pequenos (~98 bytes);
- Protocolo dominante ICMP;
- TTL mínimo igual a 63;
- Baixo volume de tráfego.

Cenário UDP (iperf)

- Pacotes significativamente maiores (~1482 bytes);
- Protocolo dominante UDP;
- Maior throughput observado.

As métricas refletem imediatamente as mudanças no padrão de tráfego, validando o mecanismo de telemetria implementado.

Além disso:

- o tráfego entre **h1** e **h2** não sofre interferência;
 - **h3** recebe exclusivamente os clones de monitoramento;
 - o dashboard atualiza os dados em tempo real;
 - os protocolos são diferenciados visualmente no dashboard.
-

10. Conclusão

O trabalho demonstrou a implementação bem-sucedida de um mecanismo de telemetria utilizando P4 e BMv2.

A solução baseada em clonagem mostrou-se:

- simples;
- eficiente;
- não intrusiva;
- totalmente implementada no plano de dados.

As métricas exportadas fornecem informações relevantes sobre o comportamento da rede e respondem rapidamente a mudanças no tráfego observado.

Além disso, a implementação evidencia o potencial da programação de planos de dados para aplicações modernas de monitoramento e observabilidade em redes programáveis.